Estensione dell'Albero di Regressione

1. In cosa consiste l'estensione

Il progetto originale dell'**Albero di Regressione** nasce come applicazione didattica basata su un'architettura Client-Server tradizionale, in cui la comunicazione avviene tramite **connessioni socket TCP sincrone e bloccanti** e l'interazione con l'utente si svolge esclusivamente da **riga di comando (console)**. In tale modello, durante la fase di predizione, il server e il client mantengono aperta una connessione continua, richiedendo all'utente di digitare numericamente le risposte da tastiera a ciascun bivio dell'albero.

L'**estensione** modernizza l'intero sistema trasformandolo in una **piattaforma applicativa distribuita e fruibile via Web**, disaccoppiata e reattiva, senza tuttavia alterare o rimuovere il funzionamento originario.

In sintesi, l'estensione consente di: - Addestrare o caricare l'albero di regressione sia attraverso servizi Web standard, sia con la classica applicazione a console. - Visualizzare la struttura dell'albero e le regole decisionali estratte sia in formato testuale, sia come **diagramma grafico visuale** direttamente nel browser. - Effettuare la predizione guidata in modalità **stateless** tramite pulsanti interattivi, eliminando la necessità di tenere thread o socket bloccati in attesa dell'input dell'utente. - Gestire, compilare, testare e avviare i moduli del progetto in modo rapido e automatizzato su ambiente Windows.

---

2. Cosa è stato aggiunto

L'estensione introduce diversi blocchi funzionali e miglioramenti:

2.1 Backend Web RESTful (Spring Boot)

Il modulo Server è stato arricchito con un'infrastruttura basata su **Spring Boot**, che affianca il server socket esistente: - **API REST su protocollo HTTP/JSON:** esposizione di endpoint standard che permettono di: - Addestrare l'albero leggendo i dati da una tabella del database relazionale MySQL. - Caricare un albero precedentemente salvato su file binario serializzato (`.dmp`). - Effettuare predizioni su nuovi esempi passo dopo passo. - **Navigazione Stateless:** il percorso decisionale viene gestito come una sequenza di scelte; ad ogni passo il server riceve il percorso percorso fino a quel momento e risponde indicando se si è raggiunta una foglia (con il valore continuo predetto) oppure il bivio successivo con le relative opzioni. Non ci sono sessioni bloccate sul server. - **Interfaccia di Collaudo Swagger UI / OpenAPI:** documentazione interattiva integrata, consultabile all'indirizzo `http://localhost:8080/swagger-ui.html`, per testare direttamente dal browser tutte le chiamate REST senza dover usare strumenti esterni. - **Sistema di Cache in Memoria:** per velocizzare l'elaborazione ed evitare accessi ripetuti al database o al disco fisso durante navigazioni successive dello stesso albero. - **Gestione Centralizzata degli Errori:** traduzione automatica delle anomalie (es. tabella inesistente, file non trovato, input non valido) in risposte d'errore chiare e comprensibili.

2.2 Interfaccia Grafica Web Reattiva (Vaadin Flow)

Il modulo Client è stato integrato con una Web Application reattiva sviluppata con **Vaadin Flow**, accessibile dal browser all'indirizzo `http://localhost:8081`: - **Visualizzazione Grafica dell'Albero (Diagramma Mermaid):** l'albero di decisione viene convertito e visualizzato dinamicamente come un diagramma vettoriale chiaro e leggibile, con nodi di divisione azzurri, rami con le relative condizioni e foglie verdi con il valore numerico predetto. - **Elenco Schematizzato delle Regole (IF-THEN):** presentazione chiara e leggibile delle regole

decisionali estratte dall'albero, strutturate a schede e organizzate con etichette visive per ogni singola condizione. - **Predizione Guidata e Intuitiva:** - Ad ogni passo della decisione compaiono pulsanti con le opzioni applicabili (es. `= A`, `<= 2.5`); - Una barra di navigazione a ritroso (**breadcrumbs**) permette all'utente di rivedere i passi compiuti e tornare indietro in qualsiasi momento con un semplice click; - Raggiunta la foglia, una scheda visiva evidenzia il valore continuo finale stimato, con opzione per avviare subito una nuova predizione. - **Apertura Automatica del Browser:** all'avvio del client, il browser predefinito di sistema viene aperto in automatico all'indirizzo della schermata iniziale.

2.3 Script Batch di Gestione per Windows (.bat)

Nella cartella principale del progetto sono stati inseriti 4 script dedicati per semplificare l'uso quotidiano: - `build.bat`**:** compila i moduli ed effettua il packaging degli archivi eseguibili in modalità silenziosa, mostrando a video solo la conferma di completamento senza schermate dispersive. - `test.bat`**:** esegue l'intera suite di test automatizzati mostrando a video un riassunto immediato per modulo ed eliminando automaticamente ogni file temporaneo. - `start_server.bat`**:** avvia il server REST su porta `8080`, mostrando un chiaro messaggio quando il servizio è pronto per ricevere richieste. - `start_client.bat`**:** avvia il client web su porta `8081`, notificando l'utente a video e aprendo in automatico la pagina nel browser.

2.4 Suite di Test Automatizzati

Per garantire che le nuove funzionalità non abbiano introdotto difetti o alterato la logica esistente, è stata implementata una suite completa di **64 test automatizzati** (JUnit 5 e Mockito): - **Test sul Server (44 test):** coprono l'induzione dell'albero, la gestione delle eccezioni, la serializzazione, i servizi REST e la navigazione passo-passo. - **Test sul Client (20 test):** validano la conversione visuale in diagrammi ad albero, il corretto tracciamento delle regole, il client HTTP e l'apertura del browser.

2.5 Conservazione e Piena Retrocompatibilità

L'aggiunta della componente Web e degli script non ha in alcun modo sostituito o compromesso il progetto originario: - I programmi a riga di comando via socket TCP (`Main` del server e `Main` del client) rimangono funzionanti e richiamabili. - Tutte le classi per l'accesso a MySQL, per la definizione degli attributi e per il calcolo della regressione continuano ad operare regolarmente in entrambi i contesti.